

Seguridad — claves embebidas en el APK de la bodycam

Preparación para la reunión con ciberseguridad · 21/08/2026 Ámbito: `BodyCamServer` (APK que corre en la unidad YIMAO W1). El APK del teléfono (`AeriaNexusPrototype`) **no está auditado aquí** y casi con seguridad comparte valores — ver preguntas abiertas.

1. Resumen en una frase

Tu compañero tiene razón, y el hallazgo es más amplio de lo que suena "una API key": en el APK hay un identificador de Agora sin ninguna autenticación detrás, y los tres canales por los que entra o sale evidencia (subida a Nexus, servidor HTTP 8080, Bluetooth) **no tienen ninguna credencial en absoluto**. El problema no es solo que haya una clave expuesta; es que en varios sitios no hay clave que exponer.

2. Qué se saca del APK, y cómo

Todo lo siguiente sale del APK compilado sin herramientas especiales — `unzip` del APK y un `grep` sobre el `classes.dex`. Lo he comprobado sobre `app-debug.apk`:

```
unzip -o app-debug.apk 'classes*.dex'
grep -ao "ff51540c357447f7bf060b3150bf6a3e" classes*.dex
→ classes4.dex:ff51540c357447f7bf060b3150bf6a3e
```

Valor	Dónde está en el código	Qué es
<code>ff51540c357447f7bf060b3150bf6a3e</code>	<code>LivestreamService.kt:16</code>	Agora App ID
<code>falcon_group_channel</code> / uid <code>9001</code>	<code>LivestreamService.kt:17-18</code>	Canal y uid fijos del livestream
<code>https://nexus.aeriaone.com/api/incidents/upload/</code>	<code>UploadService.kt:24</code>	Endpoint de subida de evidencia
<code>off-001</code>	<code>UploadService.kt:112</code>	Código de agente, fijo en el código
<code>FA1C0000-1337-4242-CAFE-DEADBEEF0001</code>	<code>BtServerService.kt:34</code>	UUID del servicio RFCOMM
Puerto <code>8080</code>	<code>Protocol.kt:23</code>	Servidor HTTP de descarga de grabaciones

Conviene llevar esta demo a la reunión: deja claro que no hace falta ser experto en reversing, y desarma de entrada cualquier propuesta del tipo "lo ofuscamos".

3. Hallazgos, por gravedad

3.1 — CRÍTICO · El livestream de Agora no tiene autenticación

LivestreamService.kt:86-88

```
// null token – only works if App Certificate is NOT enabled in Agora console.  
eng.joinChannel(null, AGORA_CHANNEL, BODYCAM_UID, options)
```

El matiz importante para la reunión: **el App ID de Agora no es un secreto por diseño** — va siempre en el cliente. Lo que lo protege es el *App Certificate*, que obliga a presentar un token firmado para entrar en un canal. Aquí está desactivado (el propio comentario lo dice), y encima el canal es uno solo, global y fijo.

Consecuencia práctica: cualquiera con ese App ID —que es cualquiera que tenga el APK— puede entrar en `falcon_group_channel1` desde la web demo de Agora y **ver y oír en directo lo que emite la bodycam**. También puede publicar vídeo falso en el canal con otro uid, o expulsar a la unidad ocupando el uid 9001.

Opciones

	Qué implica	Coste
A. Activar App Certificate + servidor de tokens (recomendada)	Nexus expone <code>POST /api/stream/token</code> autenticado; devuelve un RTC token ligado a canal, uid y rol, con TTL corto. La unidad lo pide antes de <code>joinChannel</code> y lo renueva en <code>onTokenPrivilegeWillExpire</code> .	Backend: pequeño (SDK de Agora lo genera). Unidad: medio.
B. Canal por incidente o por dispositivo	<code>falcon_<deviceId>_<ts></code> en vez de uno global. Limita el alcance y evita que dos unidades colisionen en el mismo canal.	Bajo.
C. Roles explícitos	Bodycam <code>BROADCASTER</code> , teléfono <code>AUDIENCE</code> , fijado <i>en el token</i> , no en el cliente.	Bajo, va incluido en A.

Recomendación: A + B + C, son la misma tarea. **Y rotar el App ID actual**: hay que darlo por quemado.

3.2 — CRÍTICO · La subida de evidencia a Nexus no lleva autenticación

UploadService.kt:88-160. El multipart que se envía contiene:

- `officer_code` = `"off-001"` — literal en el código
- `raw_metadata` = JSON con `device_id` (el `ANDROID_ID`), modelo y timestamp
- el fichero

Ninguna cabecera `Authorization`, ningún token, ninguna firma. Si el servidor acepta eso, **cualquiera puede subir "evidencia" a Nexus atribuida al agente off-001**, sin tener siquiera el APK: basta con ver una petición. Eso no es solo un problema de acceso, es un problema de cadena de custodia — deja de poder afirmarse que un vídeo en Nexus viene de una unidad concreta.

Es el hallazgo más grave de la lista, aunque no sea el que "parece una API key": aquí directamente no hay secreto, ni bueno ni malo.

Añadido: el `officer_code` fijo hace que todo lo que sube cualquier unidad aparezca como el mismo agente. Tiene que salir del enrolamiento del dispositivo.

Opciones

	Qué implica	Valoración
A. Identidad por dispositivo con clave en Android Keystore (recomendada)	En el enrolamiento la unidad genera un par de claves cuya privada no sale del chip, registra la pública en Nexus, y firma cada subida (JWT o mTLS). No hay ningún secreto en el APK : se genera en el dispositivo.	La correcta. Esfuerzo medio, y resuelve también quién subió qué.
B. Token de corta vida + refresh	La unidad se autentica una vez y guarda el refresh en Keystore / EncryptedSharedPreferences.	Válida; más simple que A pero el refresh sigue siendo un secreto en reposo.
C. API key por dispositivo entregada en el enrolamiento + TLS pinning	Parche rápido. Cada unidad tiene la suya, así que una filtración se revoca sola.	Peor que A, muchísimo mejor que hoy. Es la opción "para esta semana" si la reunión pide algo inmediato.

Nunca: una API key única compartida por todas las unidades y metida en el APK. Es exactamente el problema actual con otro nombre.

3.3 — ALTO · El servidor HTTP 8080 sirve toda la evidencia sin autenticación

`FileServerService.kt` — NanoHTTPD escuchando en el puerto 8080, en todas las interfaces, **arrancado en `BtServerService.onCreate()` (`BtServerService.kt:198`)**, es decir desde el arranque de la unidad y de forma permanente, no solo cuando el teléfono lo pide.

Sin token, sin TLS, sin filtro de origen:

- `GET /recordings` — lista todas las grabaciones
- `GET /recordings/{fichero}` — descarga cualquiera de ellas
- `GET /preview` y `/preview/stream` — imagen en directo mientras se graba
- `GET /benchmark?file=...` — permite lanzar trabajo de CPU y disco sobre ficheros

Cualquiera en la misma red WiFi —la de la comisaría, un hotspot público, un AP falso— se descarga la evidencia completa de la unidad.

Opciones

	Qué implica	Coste
A. Token de sesión efímero (recomendada)	Al abrir el servidor, el teléfono recibe por el enlace Bluetooth ya autenticado un token aleatorio; toda petición HTTP debe presentarlo. Barato y elimina casi todo el riesgo.	Bajo.
B. Arrancar el servidor solo bajo demanda	Hoy vive desde el boot. Que lo levante un comando del teléfono y se pare al desconectar. Reduce la ventana de exposición.	Bajo.

	Qué implica	Coste
C. Quitar <code>/benchmark</code> del build de producción	El propio código ya lo marca como herramienta de medición, no producto.	Trivial.
D. TLS autofirmado + pinning en el teléfono	Añade confidencialidad en tránsito, no solo control de acceso.	Medio.

Recomendación: A + B + C ya; D según lo que exija el cliente final.

A favor: `handleDownload` valida el `canonicalPath` contra el directorio de grabaciones (`FileServerService.kt:85`), así que **no hay path traversal**. Ese detalle está bien resuelto y conviene decirlo.

3.4 — MEDIO-ALTO · El canal Bluetooth no está autenticado

`BtServerService.kt:251`

```
ss = adapter.listenUsingInsecureRfcommWithServiceRecord(SERVICE_NAME, FALCON_UUID)
```

`Insecure` significa sin emparejamiento y sin cifrado de enlace. El UUID es fijo y se anuncia por SDP, así que no es una credencial: es una etiqueta pública.

Cualquiera a distancia de RFCOMM puede conectarse y enviar comandos en texto plano: `REC_STOP`, `PHOTO`, `STREAM_START`, `TORCH_ON`, `GPS_OFF` ... es decir, **detener la grabación de una bodycam ajena**, o encender su livestream.

Opciones

	Qué implica
A. <code>listenUsingRfcommWithServiceRecord</code> (seguro) con emparejamiento en el enrolamiento	Autenticación y cifrado a nivel de enlace BT. Hay que probarlo contra el firmware de la W1 , que es la razón probable de que hoy sea <code>insecure</code> .
B. Handshake de aplicación	Reto-respuesta HMAC con un secreto establecido en el enrolamiento, antes de aceptar comandos. Funciona aunque haya que quedarse en <code>insecure</code> por compatibilidad.
C. Lista blanca por MAC del teléfono enrolado	Barato y complementario; por sí solo no basta (la MAC se suplanta).

Recomendación: probar A; B como garantía independiente del firmware; C de propina.

3.5 — Contexto · Hoy no existe ninguna clave de cifrado de evidencia

`CryptoBenchmark.kt` genera una clave AES-256 aleatoria y la descarta: es instrumentación de medida, no producto. **El MP4 se guarda en claro** en `/sdcard/FalconOne/`, legible por cualquier app con `READ_EXTERNAL_STORAGE` y por quien tenga la unidad en la mano.

Es previsible que ciberseguridad pregunte por esto y, sobre todo, por *dónde vivirá la clave*. Conviene llegar con la respuesta, porque enlaza con la medición que ya tenemos hecha (~5,1 s por 20 min de vídeo, coste asumible).

Opciones

	Qué implica	Pega
A. Clave en Android Keystore (TEE) por dispositivo	La clave no sale del chip; no hay clave en el APK.	Si la unidad se rompe, la evidencia local es irre recuperable. Y hay que verificar el respaldo por TEE del Unisoc SC9832e con API 28.
B. Envelope encryption (recomendada)	Clave aleatoria por fichero (DEK), envuelta con la clave pública de Nexus (KEK). Solo el servidor descifra. Una clave <i>pública</i> embebida en el APK no es una filtración: es su sitio natural.	La unidad no puede reproducir lo que ya cifró.
C. Híbrido	La DEK se envuelve dos veces: con la pública de Nexus y con una clave de Keystore, para permitir reproducción local.	Algo más de complejidad; es lo que suelen hacer los productos serios.

Recomendación: B como base, C si se exige reproducción local en la unidad. B tiene además el argumento retórico perfecto para esta reunión: elimina el problema de "secretos en el APK" en vez de mitigarlo.

3.6 — BAJO · El build no está ofuscado ni configurado para release

`app/build.gradle.kts`: `isMinifyEnabled = false`, sin `signingConfig` de release, sin reglas de ProGuard. El APK analizado es con toda probabilidad el de `app/build/outputs/apk/debug/`, que además es depurable. Y `targetSdk = 28` renuncia a mitigaciones modernas y permite tráfico en claro por defecto.

R8 es el compresor/ofuscador que trae Android Studio (sucesor de ProGuard). Al compilar en modo release recorta el código que no se usa y **renombra** clases, métodos y campos a `a`, `b`, `c`... para que el APK pese menos y sea más incómodo de leer. Se activa con `isMinifyEnabled = true`.

Aviso para la reunión, importante: activar R8 **no resuelve nada de lo anterior**. R8 renombra clases y métodos; **no cifra las cadenas de texto**. El App ID, las URLs y el UUID seguirían saliendo con el mismo `grep` de la sección 2. Sirve para encarecer el análisis, no para guardar secretos. Si alguien propone "ofuscamos y listo", este es el argumento para rebatirlo.

4. Lo que sí está bien (conviene decirlo)

- `local.properties` está en `.gitignore`; no hay keystores ni ficheros de credenciales en el repositorio.
- No hay contraseñas de usuario ni tokens de terceros en el código.
- `handleDownload` valida el `canonicalPath` → sin path traversal.
- La subida a Nexus va por HTTPS.
- `allowBackup="false"` en el manifest.

- Los componentes internos son `exported="false"`; solo están expuestos `MainActivity`, `BootReceiver` (con acción protegida por el sistema) y el servicio de accesibilidad, que lo exige la plataforma. Es correcto.

5. El principio de fondo — una diapositiva

Todo lo que va dentro de un APK es público. El APK está en manos del usuario, y `unzip` + `grep` bastan para leerlo entero.

Por tanto: **en el cliente solo pueden vivir identificadores y claves públicas. Cualquier cosa que otorgue un permiso tiene que emitirla un servidor, ser de corta vida y estar ligada a un dispositivo concreto.**

Las cinco propuestas de arriba son la misma regla aplicada a cinco sitios distintos.

6. Preguntas abiertas — para responder en la reunión

1. **¿Valida algo hoy el endpoint de Nexus?** ¿Rechaza un POST anónimo? No lo he probado contra producción a propósito; hay que confirmarlo con backend.
2. **¿Está el App Certificate desactivado en la consola de Agora?** El comentario del código dice que hay que desactivarlo para que funcione, pero hay que verlo en la consola.
3. **¿Cómo se enroлан las unidades?** Es el sitio natural para instalar la credencial por dispositivo. Si no existe un proceso de enrolamiento, hay que diseñarlo: es el prerequisite de casi todas las opciones A.
4. **¿Es público el repositorio `alstar328/BodyCamServer`?** Si lo es, el App ID también está ahí y en el historial (entró en `4797b0a`). Sacarlo del código no basta: hay que **rotarlo**.
5. **¿Qué exige la cadena de custodia?** ¿Basta con confidencialidad, o hace falta que la evidencia sea *verificable* (hash firmado por la unidad)? Cambia el diseño de 3.5.
6. **¿Tiene el APK del teléfono los mismos valores embebidos?** Casi seguro comparte App ID y URL de Nexus. Hay que auditarlo igual antes de dar por cerrado esto.

7. Orden de trabajo propuesto

Priorizado por riesgo dividido por esfuerzo:

1. **Rotar el App ID de Agora, activar App Certificate y montar el servidor de tokens.** Es lo que hoy permite que un tercero vea el directo.
2. **Autenticación por dispositivo en la subida a Nexus,** y `officer_code` real del enrolamiento.
3. **Token de sesión en el 8080,** arranque bajo demanda y quitar `/benchmark`.
4. **Bluetooth emparejado o handshake de aplicación.**
5. **Cifrado de evidencia con envelope encryption** (enlaza con la medición ya hecha).

6. **R8, firma de release y subir** `targetSdk` . Al final, y sabiendo que es cosmético frente a lo anterior.

Los puntos 1 a 3 dependen de que exista un proceso de enrolamiento de dispositivos. Si no lo hay, esa es en realidad la tarea 0 y conviene sacarla de la reunión decidida.